

A Randomness Audit of the Polylog-/ORS-Update Dynamic Matching Pipelines

Task A0 technical note, v0.3 — companion to the execution plan and to `audit_lab`

Prepared for Ruifeng Cao

June 11, 2026

Abstract

Task A0 of the execution plan asks: for every random choice inside the pipelines BKS_W23, Beh23, ABR24, BG24, AKK25, determine (i) what the randomness buys, (ii) the failure mode if it is fixed, and (iii) whether limited independence suffices. This note records read-through passes 1–2 (arXiv full texts where retrievable, June 2026), the resulting machine-readable audit table (Table 1, auto-generated from `auditlib/audit_table.py`), and the experimental evidence E1–E5. The headline structural finding: once the already-deterministic layers are inventoried (boosting is *derandomized* by Tirodkar; greedy, ORS-charging, and bookkeeping carry no coins), the entire derandomization problem for the ORS-parameterized pipelines reduces to exactly *three* modules — the opportunistic sampling matcher, the Behnezhad-21 estimation interface, and randomized vertex sparsification. The estimation interface provably admits no deterministic drop-in (plan Lemma 3.6 / Cor. 3.7); the sampling matcher is a *search problem with a promise*, outside that lower bound, and is therefore the genuinely open algorithmic target (refined Task A5). Status tags [V]/[M]/[P]/[S] follow the plan.

1 Scope, sources, and verification status

The five pipelines under audit and the sources actually read in this pass:

Paper	arXiv source	Read in this pass	Residue ([M])
BG24 [4]	2404.06069v4	§1–§4.4 (main algorithm, full)	§5 (ORS upper bd.; not audit-relevant)
AKK25 [5]	2406.13573	§1–§4.1 + Lem. 2.5 proof	§4.2 recursion, §5 (overview only)
BKS _W 23 [1]	2207.07438v3	abstract; App. A Prop. A.1; JACM §2/§5	§5+ robustness lemmas
Beh23 [2]	2207.07607v2	abstract + §1 technique overview	body (EDCS characterization etc.)
ABR24 [3]	2307.08772v1	§1–§4 (Thm 1: adaptive, worst-case [V])	§5 dynamic details
Beh21 [6]	2106.02942v3	§1–§3 + §5 statements	§3.1–3.2 proof internals

Also verified at statement level: Kiss’22 vertex sparsification in its operative form, BKS_W23 Prop. A.1 (“a randomized algorithm which for each update to G makes an update to $O(\log^2 n/\varepsilon^3)$ contracted subgraphs, such that w.h.p. . . .”); Tirodkar’s derandomization of the McGregor boosting framework as stated in AKK25 (Prop. 2.2); and [10] Thm. 6 ($\Omega(n^2)$ queries to *find* a maximal matching, even randomized). Remaining [M] residue: BDHSS19/Sol16 maintenance internals, BKS_W23 §5+ robustness lemmas, Kis22 full internals.

Remark 1 (Version numbering). *AKK25* cites “[BG24, Lemma 9]” and “[BG24, Lemma 14]”. In the *v4* text those statements are Lemma 7 (*RandomSampling*) and Lemma 11 ($\sum_i 1/d_i \leq O(\alpha^{-2} \text{ORS}_n(r(1-\alpha)) \log n)$). All BG24 references below are to *v4*.

2 Pipeline anatomy: where the coins actually live

Read-through pass 1 supports the following layered decomposition. The right-hand column is the verified location of the claim.

Layer	Coins?	Verified location
Approximation boosting (McGregor framework)	no	AKK25 Prop. 2.2: “de-randomized in [Tir18]”
Greedy matching on explicit edge lists	no	AKK25 Fact 2.1; BG24 GREEDYMATCHING (§4.1)
ORS charging / amortization	no	BG24 Lem. 11; AKK25 Lem. 2.5 (its probabilistic method is <i>analysis-only</i>)
Certificate / batch bookkeeping (H_{cert} ; G_{old} , G_{batch} , G_{match})	no	BG24 Alg. 2; AKK25 Alg. 2 + Fig. 1
Bounded-degree $(1 + \varepsilon)$ maintenance and the periodic-recomputation paradigm (Gupta–Peng)	no	deterministic; BLM20 Lem. 6.1; BKS23 JACM §2
Opportunistic sampling matcher	yes	BG24 Alg. 1/Lem. 7 + Claim 8; AKK25 Alg. 1/Lem. 3.1 + Claims 3.3–3.4
Sublinear size-estimation interface (Beh21)	yes	BG24 Prop. 4; AKK25 Prop. 2.3; BKS23 abstract (white-box 2nd pass); Beh23 §1; ABR24 §1
Random ranks / RGMM (inside Beh21; phase-1 of Beh23/ABR24)	yes	ABR24 §1: maximal matching “using the fast algorithms of [BGS, Sol16, BDHSS19]”
Vertex sparsification (additive→multiplicative)	yes	BKS23 Prop. A.1 ($O(\log^2 n/\varepsilon^3)$ random contracted subgraphs, whp); AKK25 Prop. 2.4; BG24 Prop. 5; Beh23 §1
Adaptivity wrapper (fresh coins per rebuild; low-sensitivity outputs)	yes	BKS23 abstract; BG24 §3 + Alg. 2 ll. 17–20 + Lem. 9; AKK25 Result 1 + Problem 1

Two consequences, both load-bearing for WS-A:

1. **The deterministic frontier has exactly three modules.** Everything above the midrule needs no derandomization. What remains is: (a) the opportunistic sampler (rows R4/R5 of Table 1); (b) the Beh21 estimation interface (R1/R2); (c) vertex sparsification (R7).
2. **BG24’s coins are narrower than the plan’s v0.1 audit assumed.** The ORS-epoch amortization itself (Lemma 11, H_{cert}) is deterministic combinatorics; the randomness of BG24 lives *only* in Lemma 7’s sampling, plus the R1/R7 interfaces and the per-rebuild freshness. This sharpens plan row R4 accordingly.

3 The audit table

Table 1 is generated from the machine-readable source (`auditlib/audit_table.py`) by `run_all.py`; the `.tex` fragment below is `results/audit_table.tex` verbatim.

Table 1: Randomness audit (Task A0, v0.2). Tags: [V] verified against source, [M] internal reference pending, [P] proved in the plan, [E:*] empirical evidence in this package.

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in-dep.	Evidence
R1	Sublinear Monte-Carlo matching-size estimation via adjacency-matrix queries (Beh21). Verified statements: BG24 Prop. 4; AKK25 Prop. 2.3 (Otilde(n) poly(1/eps)) queries, whp $0.5\mu - \epsilon n \leq \text{est} \leq \mu$; BKSW23 abstract ('invoke ... Behnezhad (FOCS'21) in a white-box manner' for the second streaming pass); Beh23 SS1 ('key tool ... estimating the size of maximal matchings in Otilde(n) time'); ABR24 SS1 (phase 2 'augment ... by employing the sublinear-time matching size estimator of [Beh21]'). (all five (engine: Beh21))	Per-rebuild Otilde(n) cost with whp additive accuracy; the only access to the (kernel/contracted/leftover) graph is via probes.	Adversary who knows the fixed probe/sample set plants all matching mass outside it: estimator reports ~ 0 while $\mu \geq n/4$. Provably no deterministic probe algorithm with $o(n^2)$ probes exists (plan Lemma 3.6); no drop-in derandomisation of this interface (plan Cor. 3.7).	Chebyshev-level accuracy needs only pairwise sample choices; whp needs $k = \Theta(\log n)$ -wise tails. Whether the surrounding union bounds tolerate the weaker tails is Task A4. Beh21-internal requirements not yet re-read.	E1 (works); E5 (fixed fails); Lemma 3.6; Cor 3.7 [V][P][E:e1,e5]

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in- dep.	Evidence
R2	Uniformly random edge/vertex ranks driving greedy maximal matching: (a) inside the Beh21 estimator (local RGMM oracle); (b) as the explicitly maintained phase-1 maximal matching of the size pipelines – ABR24 SS1: 'maintain a maximal matching explicitly using the fast algorithms of [BGS, Sol16, BDHSS19]' (randomized, oblivious-adversary maintenance). (Beh21 inside all five; phase-1 of Beh23/ABR24)	Verified (Beh21 Thm 3.5): $E[T(v, \pi)] = O(\bar{d} \log n)$ over uniformly random v AND uniformly random π , via the directional oracles VO/EO (Beh21 Algs 1-2, Remark 3.3); caching gives $\text{poly}(n)$ worst case with prob. 1 (Remark 3.2, Obs 3.7). In ABR24 the random order is ADDITIONALLY load-bearing for the approximation analysis of the b-matching (even-spread-of-copies argument, ABR24 SS2), not only for locality.	Rank-aware adversary builds a rank-decreasing path: one oracle query costs $\Theta(m)$ (E2); head-edge toggling forces $\Theta(n)$ recourse per update (E4). Approximation is NOT what breaks (any maximal matching stays $1/2$).	VERIFIED OB- STRUCTION (Beh21 SS3): the charging proof of Lemma 3.10 maps a permutation π to $\phi(\pi, P)$ by rotating ranks along query paths – a bijection of the FULL symmetric group. k -wise families are not closed under these rotations, so the $O(\bar{d} \log n)$ locality analysis does not transfer to limited independence as-is (Task A4 now has a precise technical obstacle). Empirically (E3): even $k=2$ matches full independence on oblivious instances – a genuine analysis/experiment gap. BDHSS19 maintenance internals unread [M].	E2 ($\Theta(m)$ blowup); E4 ($\Theta(n)$ recourse); E3 (k -wise) [V][M][E:e2,e3,e4]

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in- dep.	Evidence
R3	Adaptive-adversary robustness wrappers. Verified claims: BKS23 abstract (adaptive + worst-case polylog, both whp); BG24 SS3 (adversary sees previous outputs) + Thm 1; AKK25 Result 1 + Problem 1 (adaptively chosen queries); Beh23 SS1 (vertex sparsification 'can be made work against adaptive adversaries too [Kis22]'); ABR24 Thm 1 ('poly(log n) worst-case update time even against adaptive adversaries', via LOCAL-model simulation on the bounded-degree $G[M \cup B]$ + size-only output); BKS23 worst-case via deamortization (JACM SS2: 'easily deamortized'). Mechanism: low-sensitivity outputs (sizes; or an explicit matching only shrunk by deletions between rebuilds, BG24 Lemma 9) + fresh coins per rebuild/call + whp union bounds over poly(n) updates. (BKS23; Beh23; ABR24; BG24; AKK25)	Correctness whp against adversaries that see the outputs.	With fixed coins the transcript determines internal randomness; every adversary is effectively adaptive against a deterministic algorithm – OP-D has no 'oblivious deterministic' easy mode (plan SS2.1, Obs 3.4).	Across rebuilds: coins must be fresh/independent. Within a rebuild: inherits from R1/R2/R4/R5.	conceptual; plan SS2.1, Obs. 3.4; BG24 Alg 2 ll.17-20 [V][M]
R4	RandomSampling (BG24 Algorithm 1, Lemma 7): per-vertex uniform sampling of candidate partners with geometrically doubling budgets $b = n^{\{2\delta(i-1)\}}$; whp degree bound via Claim 8 (per-(v,i) failure $1/n^3$, union bound). Fresh coins per call; called from MatchAndCertify (Lemma 6) inside Algorithm 2 every eps $n/2$ updates. (BG24)	Finds an $\Omega_{\text{eps}}(n)$ matching in $G[U]$ in $\tilde{O}(n^{\{2+\text{eps}\}}/d)$ where $d = \text{avg internal degree of the certificate}$ – fast exactly when the instance is far from induced.	Adversary aligns the instance with the fixed sample schedule: planting off-the-samples (E5 mechanism) breaks the Claim-8 degree invariant. NOTE (verified): the ORS charging itself (Lemma 11), H_{cert} bookkeeping, and the boosting wrapper (Prop. 3) are deterministic – the coins of BG24 live ONLY in Lemma 7 + the R1/R7 interfaces.	Claim 8 needs $(1-p)^{\{\Omega(\log n / p)\}} \leq n^{\{-\Theta(1)\}}$ tails: $\Theta(\log n)$ -wise independence plausibly suffices (Bonferroni truncation) – unproven, input to Task A4 [S].	E5 (mechanism); BG24 Lem 7, Claim 8, Lem 11, Alg 2 [V][E:e5]

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in- dep.	Evidence
R5	Opportunistic sublinear matcher (AKK25 Algorithm 1, Lemma 3.1): per-vertex neighbour sampling with geometric probabilities $p_i = n^{\{3 \gamma\}}/n$ (binomial-count implementation, fn.1); whp runtime via Claim 3.3 (Chernoff) and max-internal-degree via Claim 3.4 ('residual sparsity' + union bound). Used inside the three-graph batch scheme ($G_{\text{old}}/G_{\text{batch}}/G_{\text{match}}$), - FINDING a Alg 2, Lemma 4.1) and recursed via the family $\{A_i\}$ (SS4.2). (AKK25)	Strengthens BG24 Lemma 7: $O(\tilde{m} n^{\{3 \gamma\}}/\Delta_{\text{IN}}(M))$ time, sensitive to $ E $ and to MAX (not avg) internal degree - this is what removes the $\sqrt{n}$ factor. Worst-case $\Omega(m)$ is necessary: ACK19 (Assadi-Chen-Khanna SODA'19) Thm 6, - FINDING a maximal matching needs $\Omega(n^2)$ queries even for randomized algorithms.	Same plant-off-the-samples class as R4; the recursion compounds the exposure (each level reruns the sampler). Deterministic route expectation unchanged: BG24 shape first (plan Task A5, [C]).	Claims 3.3/3.4 are Chernoff + union bound: $\Theta(\log n)$ -wise plausibly suffices [S]; recursion multiplies the number of seeds, not their required strength. SS4.2/SS5 details read at overview level only.	AKK25 Lem 3.1, Claims 3.2-3.4, Lem 4.1; E5 (mechanism) [V][M]
R6	Whp amplification: union bounds over $\text{poly}(n)$ updates / calls (explicit in BG24 Claim 8 and AKK25 Claims 3.3-3.4); fresh repetition where a single run succeeds with constant probability. (all five)	High-probability correctness, union-boundable over time.	No deterministic analogue inside the probe interface (plan Lemma 3.6 kills the single run, so repetition cannot help); outside it, replacement = maintained deterministic summaries (plan WS-A).	Repetitions must be mutually independent.	Lemma 3.6; Cor 3.7 [P]

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in- dep.	Evidence
R7	Randomized vertex sparsification for the additive-to-multiplicative reduction: contract vertices into $O(\mu/\epsilon)$ groups preserving $(1-\epsilon)$ of the matching, whp [AKLY16, AKL16, CCE+16] as implemented dynamically by Kis22. Verified statements: BKS23 Prop. A.1 (App. A): 'a randomized algorithm which for each update to G makes an update to $O(\log^2 n / \epsilon^3)$ contracted subgraphs, such that w.h.p. ...'; AKK25 Prop. 2.4; BG24 Prop. 5; Beh23 SS1 (adaptive via [Kis22]). (Beh23; BG24 (Prop. 5); AKK25 (Prop. 2.4); via Kis22)	Reduces vertex count to $O(\mu/\epsilon)$, turning ϵ n-additive guarantees into $(1-\epsilon)$ -multiplicative with polylog overhead, whp, including against adaptive adversaries.	A fixed (published) contraction partition lets the adversary place the matching across collision classes, destroying preservation – same plant-against-the-known-seed pattern as E5, one level up.	Preservation proofs bound per-vertex collision events; pairwise independence gives the expectation, whp needs stronger tails. Kis22 full internals unread [M] (its operative form, BKS23 Prop. A.1, is verified).	E5 (pattern); read-through pending for Kis22 lemmas [V][M]

Id	Mechanism (papers)	(i) Buys	(ii) Failure if fixed	(iii) Limited in- dep.	Evidence
R8	ALREADY-DETERMINISTIC layers, verified: McGregor boosting derandomized by Tir18 (explicit in AKK25 Prop. 2.2); greedy matching (AKK25 Fact 2.1, BG24 GreedyMatching); ORS charging (BG24 Lemma 11; AKK25 Lemma 2.5 – its probabilistic method lives in the ANALYSIS only, no algorithmic coins); H_cert / G_old-G_batch-G_match bookkeeping (BG24 Alg 2; AKK25 Alg 2); Gupta-Peng bounded-degree (1+eps) maintenance (deterministic, cited e.g. BLM20 Lemma 6.1); the Gupta-Peng periodic-recomputation paradigm itself (BKSW23 JACM SS2); and – notably – ABR24 Thm 2: a DETERMINISTIC two-pass semi-streaming (2-sqrt2-eps)-approximation, showing the combinatorial core (maximal matching + duplicate-free b-matching + blossom-inequality analysis) is already deterministic under pass access; the dynamic version’s coins are forced purely by the probe access model. (inventory row (not a randomness site))	– (no coins; listed to delimit the derandomization frontier)	n/a	n/a	Frontier consequence: derandomizing BG24/AKK25 reduces to exactly three modules: the opportunistic sampler (R4/R5), the Beh21 estimator interface (R1; provably no probe drop-in), and vertex sparsification (R7). The sampler is a SEARCH problem with a promise – outside Lemma 3.6’s decision lower bound – hence the genuinely open algorithmic target (refined Task A5). [V]

4 The deterministic frontier, refined

Module (b) is closed at the interface. The estimation interface consumes the kernel/contracted/leftover graph *only through probes* (adjacency-matrix queries in BG24 Prop. 4 / AKK25 Prop. 2.3). Plan Lemma 3.6 shows that any *deterministic* adaptive probe algorithm distinguishing $\mu = 0$ from $\mu \geq n/4$ needs more than $\binom{\lceil n/2 \rceil}{2} - 1$ probes, against $\tilde{O}(n)$ randomized (Prop. 2.3); plan Cor. 3.7 concludes there is no deterministic drop-in for this layer, and experiment E5 (§5) realizes the adversary constructively. Derandomization must therefore *replace* the interface with maintained

deterministic summaries (plan Tasks A1–A3), not simulate it.

Module (a) is open — and is now a precise target. The opportunistic sampler solves a *search problem with a promise*:

Problem 1 (Det-Opportunistic-Matching; refined Task A5 target). *Given adjacency-matrix (BG24) or adjacency-list (AKK25) access to G , a set $U \subseteq V$ with the promise $\mu(G[U]) \geq \delta n$, and $\gamma < 1/6$: deterministically output a matching $M \subseteq G[U]$ with $|M| \geq \gamma \delta n$ in time $O(m \cdot n^{O(\gamma)} / \Delta_{\text{IN}}(M))$ (AKK25 shape; $\tilde{O}(n^{2+O(\gamma)} / d_{\text{avg}})$ for the BG24 shape), where $\Delta_{\text{IN}}(M)$ is the maximum degree of $G[V(M)]$.*

Three observations delimit Problem 1: (1) it is *not* covered by plan Lemma 3.6 — that bound is for a *decision* task with no promise, whereas here a large matching is promised to exist and the runtime is *allowed* to degrade to $\tilde{O}(m)$ exactly when the output is near-induced ($\Delta_{\text{IN}} = O(1)$); (2) the worst case $\Omega(m)$ is necessary even for randomized algorithms (AKK25 quote ACK19), so only the d -adaptivity needs replication; (3) the randomized solutions get d -adaptivity from one Chernoff-plus-union-bound argument (BG24 Claim 8; AKK25 Claim 3.4 “residual sparsity”), which suggests two deterministic routes worth pricing in WS-A: degree-bucketed exhaustive scanning with a potential-function charging in place of Claim 8, and expander/disperser-based sample schedules (plan Task A4 [C]). If Problem 1 is solved in the BG24 shape, the remaining obstacles to a deterministic $\tilde{O}(\sqrt{n^{1+\varepsilon}} \text{ORS})$ bound are exactly modules (b), (c) — this is the concrete content of plan Task A5’s expectation that the deterministic route lands at the BG24 shape first.

Related separations (prior-art check for the C1 note). Two published results already separate determinism from randomness for the *finding* task: Assadi–Solomon [11] (Thm. 3: deterministic $\Omega(n^2)$ vs. randomized $\tilde{O}(n\beta)$ for finding a maximal matching on adjacency-array graphs of neighborhood independence $\beta = 2$) and [10] (Thm. 6: finding a maximal matching needs $\Omega(n^2)$ queries even randomized). Plan Lemma 3.6 is a different statement — adjacency-*matrix* probes, size-*estimation* (where randomized is $\tilde{O}(n)$ [6], unlike finding), explicit clique-coverage threshold, pipeline corollary — but the C1 note must cite both prominently and scope its novelty accordingly; the plan (v1.1, Task C1(a)) now records this.

Limited independence: a verified obstruction (Task A4). Beh21’s locality bound (Thm. 3.5: $\mathbb{E}[T(v, \pi)] = O(\bar{d} \log n)$ over uniform v and uniform π) is proved by the charging map $\phi(\pi, \vec{P})$ of Lemma 3.10, which rotates ranks along query paths — a bijection of the *full* symmetric group. k -wise independent rank families are not closed under these rotations, so the analysis does not transfer to limited independence as-is. Experiment E3 nonetheless finds even 2-wise ranks empirically indistinguishable from full independence on oblivious instances: Task A4 now has both a precise technical obstacle and an experimental signal that the obstacle may be an artifact of the proof. A second nuance (ABR24 §2): in ABR24 the random order is additionally load-bearing for the *approximation* analysis of the b -matching (even-spread-of-copies argument), not only for locality — a derandomization must replace that too.

A deterministic data point (ABR24 Thm. 2). ABR24’s two-pass semi-streaming $(2 - \sqrt{2} - \varepsilon)$ -approximation is *deterministic*: the combinatorial core (maximal matching + duplicate-free b -matching + blossom-inequality analysis) needs no coins when the edges can be read in passes. The dynamic version’s randomness is forced purely by the probe access model — exactly the localization that Cor. 3.7 predicts.

Module (c). Vertex sparsification (R7) is the same plant-against-a-known-seed pattern one level up: a fixed contraction partition can be defeated by an adversary routing the matching across collision classes. Its operative statement is now pinned (BKSW23 Prop. A.1: $O(\log^2 n/\varepsilon^3)$ random contracted subgraphs per update, whp); deterministic replacement candidates are degree-based (non-contractive) vertex reductions.

5 Experimental evidence (package audit_lab)

All numbers: full scale, seed 20260611; reproduce with `python3 run_all.py -scale full`; tests T1–T6 cross-validate every measured quantity (Blossom vs. bitmask DP; local oracle vs. global greedy; construction validity). Mapping E→rows is in Table 1.

- **E1 (R1, what sampling buys).** $n=4000$, $\bar{d}=5$: vertex-sampling estimate of $|\text{RGMM}|$ has mean error $0.0024n$ at $s=6400$ samples (scaling $\propto 1/\sqrt{s}$, Fig. 1), at 0.8 oracle evaluations per vertex query; Blossom-checked $|\text{MM}|/\mu \in [0.805, 1]$.
- **E2 (R2, fixed ranks, locality).** On a rank-decreasing path with $m=64,000$ edges, *one* oracle query costs exactly m evaluations; fully random ranks: mean 1.8 (Fig. 2).
- **E3 (R2/A4, limited independence).** k -wise ranks with $k \in \{2, 3, 4, 8\}$ empirically match full independence in accuracy (err/ n : 0.0051 vs. 0.0051) and locality on oblivious instances. Empirical only; consistent with the $\Theta(\log n)$ -wise sufficiency conjecture of rows R4/R5 [S].
- **E4 (R2 dynamic).** Head-edge toggling on a 8000-edge path: fixed-rank greedy suffers 8000 recourse per update (full flip); random ranks, same oblivious updates: mean 1.20 (6667×). Supports plan Task A3(b).
- **E5 (R1/R4/R7, fixed samples; Lemma 3.6 companion).** $n=96$: against a fixed pseudorandom probe set, the constructive adversary plants $n/4$ matching edges the detector *never* hits (0 hits at every budget), while fresh random probes with budget $B=1127 = \binom{48}{2} - 1$ detect with $p = 0.993$. Extra finding: the pseudorandom family still fails at B equal to 90% of all pairs — a fixed probe set is defeated by volume-free structure, and must instead clique-cover $\lceil n/2 \rceil$ vertices; randomness is being purchased *instead of* structured coverage.

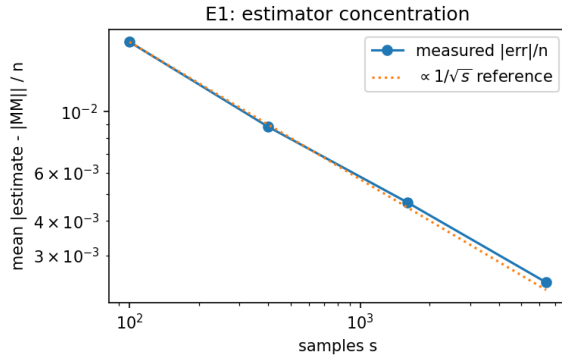


Figure 1: E1: estimator concentration.

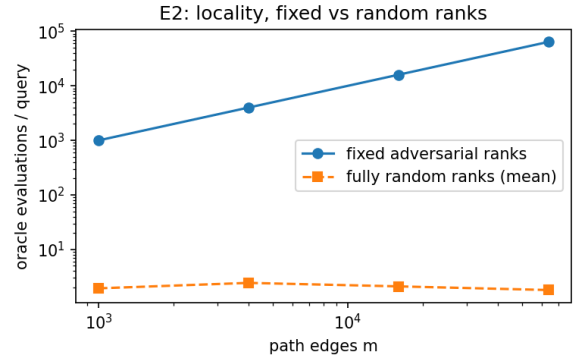


Figure 2: E2: locality, fixed vs. random ranks.

6 Status and next steps

Done (this pass). Lemma-level locations for BG24 (complete for the algorithmic part) and AKK25 (§1–4.1); abstract/overview-level verification for BKS23, Beh23, ABR24; rows R1–R8 of Table 1; experiments E1–E5 with cross-validation; the three-module frontier and Problem 1.

Done in pass 2. (1) Beh21 internals read (§1–3): the rank-rotation obstruction for Task A4 is now precise; (2) BKS23 Prop. A.1 pinned (vertex sparsification, R7); (3) ABR24 adversary model verified (Thm. 1: adaptive, worst-case) and §1–4 read, yielding the RGMM-for-approximation nuance and the deterministic-streaming data point; (4) [10] resolved (Thm. 6) and [11] prior art folded into the C1 scoping; (5) the v0.3 table and these findings are folded into the plan document, re-issued as v1.1 (Appendix A there).

Remaining (small). BDHSS19/Sol16 maintenance internals (R2 residue); BKS23 §5+ robustness lemmas (R3 residue); Kis22 full text (R7 residue — operative form verified). These do not block WS-A start.

References

- [1] S. Bhattacharya, P. Kiss, T. Saranurak, D. Wajc. Dynamic matching with better-than-2 approximation in polylogarithmic update time. SODA 2023, pp. 100–128; J. ACM 71(5):33, 2024. arXiv:2207.07438.
- [2] S. Behnezhad. Dynamic algorithms for maximum matching size. SODA 2023, pp. 129–162. arXiv:2207.07607.
- [3] A. Azarmehr, S. Behnezhad, M. Roghani. Fully dynamic matching: $(2 - \sqrt{2})$ -approximation in polylog update time. SODA 2024, pp. 3040–3061. arXiv:2307.08772.
- [4] S. Behnezhad, A. Ghafari. Fully dynamic matching and ordered Ruzsa–Szemerédi graphs. FOCS 2024, pp. 314–327. arXiv:2404.06069 (v4).
- [5] S. Assadi, S. Khanna, P. Kiss. Improved bounds for fully dynamic matching via ordered Ruzsa–Szemerédi graphs. SODA 2025, pp. 2971–2990. arXiv:2406.13573.
- [6] S. Behnezhad. Time-optimal sublinear algorithms for matching and vertex cover. FOCS 2021, pp. 873–884.
- [7] P. Kiss. Improving update times of dynamic matching algorithms from amortized to worst case. ITCS 2022.
- [8] S. Tirodkar. Deterministic algorithms for maximum matching on general graphs in the semi-streaming model. FSTTCS 2018.
- [9] A. McGregor. Finding graph matchings in data streams. APPROX–RANDOM 2005, pp. 170–181.
- [10] S. Assadi, Y. Chen, S. Khanna. Sublinear algorithms for $(\Delta + 1)$ vertex coloring. SODA 2019, pp. 767–786. (Thm. 6: finding a maximal matching needs $\Omega(n^2)$ queries, even randomized; quoted in AKK25 for the $\Omega(m)$ worst-case necessity of its Lemma 3.1.)
- [11] S. Assadi, S. Solomon. When algorithms for maximal independent set and maximal matching run in sublinear time. ICALP 2019, 17:1–17:17. (Thm. 3: deterministic $\Omega(n^2)$ for finding a maximal matching at neighborhood independence $\beta = 2$.)
- [12] M. Gupta, R. Peng. Fully dynamic $(1 + \varepsilon)$ -approximate matchings. FOCS 2013, pp. 548–557.
- [13] S. Behnezhad, J. Łącki, V. Mirrokni. Fully dynamic matching: beating 2-approximation in Δ^ε update time. SODA 2020, pp. 2492–2508.
- [14] R. Cao (prep.). Optimal approximation for fully dynamic matching and ordered Ruzsa–Szemerédi graphs: a feasibility-checked execution plan. Working document, June 2026 (Lemma 3.6, Cor. 3.7, Tasks A0–A5).